CS 4398 Software Engineering

Fall 2025

Movie Database Application

Software Requirement Specification

Group 1 Team Members:

Nicholas Bacakos

Tushig Battulga

Carolina Banda

Ananta Aryal

Table of Contents

1. Introduction 3

1.1 Document Purpose 3

1.2 Product Scope 3

1.3 Intended Audience 4

1.4 Definitions, Acronyms, Abbreviations 4

1.5 Overview/Document Conventions 4

2. Overall Description 5

2.1 Document Perspective 5

2.2 Document Functions 5

2.3 User Characteristics 5

2.4 User Interface 5

2.5 End- User Operating Environment 5

2.6 Design and Implementation constraints 6

3. Functional Requirements 7

3.1 Summary 7

3.2 Use Case Diagram 8

3.2.1 Use Case: Prompt Input & Search Trigger 9

3.2.2 Use Case: NLP → Structured Query (OpenAI) 9

3.2.3 Use Case: Movie DB Search 10

3.2.4 Use Case: Ranking & Top-N Results 10

3.2.5 Use Case: Result Cards & Detail View 11

3.2.6 Use Case: Favorites 12

3.2.7 Use Case: Recommendations from Favorites 12

3.2.8 Use Case: Availability Badges 13

3.2.9 Use Case: Theme Switch 13

3.3 Wireframe 14

3.3.1 Login Screen 14

3.3.2 Main Menu 15

3.3.3 Search Results Page 16

3.3.4 Movie Detailed View 17

3.4 Hardware/Software Requirements 18

3.4.1 Hardware Requirements for User 18

3.4.2 Software Requirements for User 18

3.4.3 Hardware Requirements for Developer 18

3.4.4 Software Requirements for Developer 18

4. Non-Functional Requirements 19

4.1 Reliability 19

4.2 Robustness 19

4.3 Performance 19

4.4 Maintainability 19

4.5 Security 19

4.6 Usability 19

5. Diagrams 20

6. References 22

1. Introduction

1.1 Document Purpose

The Movie Database Application is designed to be a modern, AI-powered tool that allows users to find movies using intuitive, natural-language searches. The system is intended to be a comprehensive solution for movie discovery and tracking. The system's main functions are to:

1.2 Product Scope

This document provides the scope of the Movie Database Application. The system's main functions are to:

- **AI Query Understanding**: Allow a user to enter a natural-language prompt to describe the kind of movie they want to watch. The system will then parse this input using an NLP API (like OpenAI) to create a structured search query with fields such as genre, era, mood, actors, and directors.

- **Database Search and Ranking**: Search a movie database using the structured query and return a list of top matches. Results will be ranked by default based on rating and popularity, with recency used as a tie-breaker.

- **Database Search and Ranking**: Search a movie database using the structured query and return a list of top matches. Results will be ranked by default based on rating and popularity, with recency used as a tie-breaker.

- **Recommendation Engine**: Generate personalized recommendations based on the user's "Favorites". The system will compute and display similar titles by analyzing genre overlap between movies.

- **Availability & Showtimes**: Integrate with streaming service data to display availability badges or "where to watch" links on movie result cards when possible. The section will be hidden gracefully if provider data is unavailable.

- **Multi-Platform Support**: Be accessible via a responsive design for both mobile and desktop web browsers.

- **User Interface**: Display movie results in clear, clickable cards that open a more detailed view with a full synopsis, cast, and rating information. The interface will also support a light/dark theme toggle that can be persisted for the user.

- **Accessibility & Localization**: Include features to improve the user experience for a wider audience, such as screen reader support, multi-language options, and filters for regional availability.

1.3 Intended Audience

The intended audience for this application includes movie enthusiasts and casual viewers who want a more intuitive and powerful way to discover films. It is designed for users who want to move beyond simple genre filters and find movies based on mood, themes, or conversational descriptions. This application will appeal to individuals who appreciate personalized recommendations and want a simple way to manage a list of films they wish to watch. Users are expected to have basic familiarity with operating a web browser on a desktop or mobile device.

1.4 Definitions, Acronyms, Abbreviations

Terms	Definition
API	Application Programming Interface: A set of rules that allows different software systems to communicate, such as this application communicating with the OpenAI API to process prompts.
DB	Database: An organized collection of movie data that the application searches to find matches for user queries.
Jaccard Similarity	A statistical measure used to gauge similarity between two sets.
NLP	Natural Language Processing: The technology that allows the system to understand and interpret a user's conversational search prompt and convert it into a structured query.
Structured Query	A machine-readable query with specific fields (e.g., genres, years, mood etc) extracted from a user's natural language prompt via the OpenAI API.
UI	User Interface: The visual elements and controls of the application that users interact with.

1.5 Overview/Document Conventions

This document describes the structure and functionality of the Movie Database Application. It presents the application's features, user interactions, and technical requirements in a clear and organized format, following standard software engineering conventions.

2. Overall Description

2.1 Document Perspective

The Software Specification document outlines the functional and non functional requirements for the Movie Finder and Recommendation. The system focuses on helping users search for movies and series using natural language prompts, browse results from and integrated movie database, and build a personalized list of favorites.Based on user preferences, the application generates recommendations to improve content discovery. The application is designed primarily as a web-based system, optimized for both desktop and mobile browsers.

2.2 Document Functions

This document serves as a guide for developers to implement core features such as:

- Parsing user prompts into structured queries using AI (OpenAI API)
- Fetching movie/series metadata from external databases
- Displaying search results with availability and genre details
- Allowing users to save favorites, maintain history, and receive recommendations
- Providing optional features such as light/dark mode and exportable favorites.

This ensures stakeholders understand what the system will provide and what outcomes should be expected at delivery.

2.3 User Characteristics

The target users are general entertainment consumers, looking for a more intuitive way to discover new content beyond genre filters. Users are expected to be familiar with using web applications and comfortable writing descriptive search queries. No specialized technical knowledge is needed beyond basic web browser operation.

2.4 User Interface

The web interface will include:

- A search bar for natural-language prompts.
- A result grid with posters, titles, and genres.
- Buttons for adding/removing favorites.
- A recommendations panel based on favorite selections.
- Light/Dark mode toggle for accessibility.
- Search history with quick re-run capabilities

2.5 End- User Operating Environment

The application is cross platform and is intended to run on modern web browsers(e.g, Google Chrome, Mozilla, Firefox, Apple Safari, and Microsoft Edge) and:

- Desktop computers.
- Mobile Devices with active internet connection.

2.6 Design and Implementation constraints

- A steady internet connection is required for all core features, as the system relies on real-time communication with external APIs.
- Navigation should support mouse,keyboard, and touch input.
- User authentication must comply with secure login standards.
- Search result speed is directly dependent on the latency of the external APIs

3. Functional Requirements

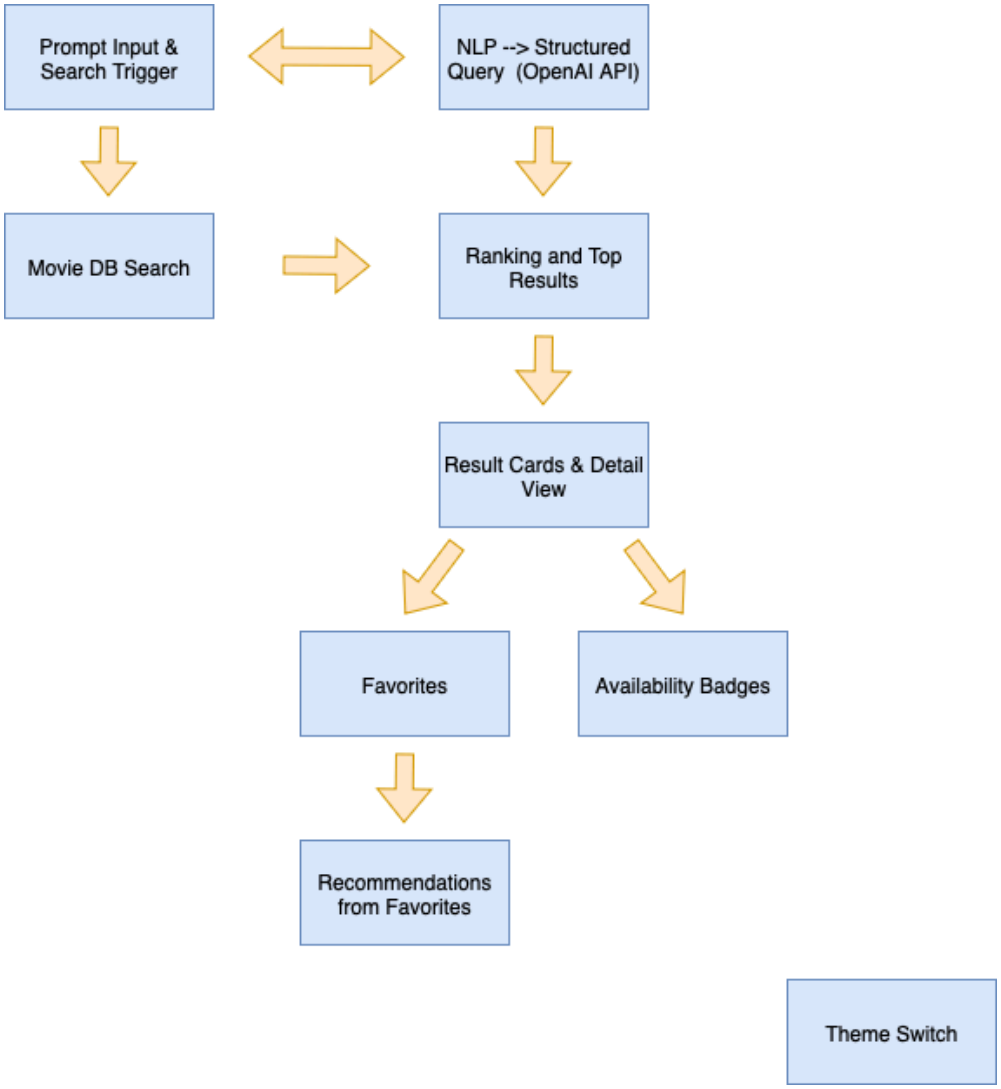
3.1 Summary

- The web app lets a user type a natural-language prompt (e.g., “feel-good 90s rom-com with more than 20 episodes and a road trip vibe”), converts it to a structured query via the OpenAI API, searches a movie DB, returns the top matches, and lets the user save picks to Favorites. Based on favorites, the app recommends similar titles (simple genre-overlap method to start). Optional niceties: availability badges, history, export, and light/dark theme.

Tech stack assumptions: React (web) + Node.js API + backing store (Postgres or MongoDB).

- Core FR IDs (high level)
- **FR-1 Prompt Input & Search Trigger**
  User enters a natural-language prompt; can hit Enter or click “Search.” Persist raw prompt.
- **FR-2 NLP → Structured Query (OpenAI)**
  Send prompt to OpenAI to extract: genres, era/years, mood/keywords, languages, runtime range, actors/directors (optional), popularity/rating constraints. Log the structured query for debugging.
- **FR-3 Movie DB Search**
  Query local DB (seeded from MovieLens/TMDb/IMDb) using the structured fields. Return list with: title, year, poster, overview, genres, average rating, popularity, runtime, languages.
- **FR-4 Ranking & Top-N Results**
  Rank results (default by rating then popularity; tie-break by recency). Return **Top 5**. Allow a “Show more” pagination.
- **FR-5 Result Cards & Detail View**
  Display movie cards; clicking opens a detail drawer/page with synopsis, genres, rating, runtime, cast snapshot (if available), and “Add to Favorites.”
- **FR-6 Favorites**
  Users can add/remove movies to a Favorites list. Show a Favorites page with sorting (by added date, rating, year) and filtering (genre).
- **FR-7 Recommendations from Favorites (Simple Similarity)**
  For each favorite, compute top 3 similar titles using **genre overlap** (Jaccard similarity). Aggregate across favorites; dedupe; show **Top 3** overall recommendations. (Stretch: include cast/crew/keywords similarity.)
- **FR-8 Availability Badges (Optional/MVP-lite)**
  If availability data is seeded (or fetched), show streaming/service badges or “where to watch.” Fallback: hide section gracefully.
- **FR-9 Theme Switch**
  Light/dark theme toggle, persisted per user/profile or device.

3.2 Use Case Diagram



3.2.1 Use Case: Prompt Input & Search Trigger

Field	Content
NAME OF THE USE CASE	Prompt Input & Search Trigger
Brief Description	User enters a natural-language prompt (e.g., “feel-good 90s rom-com with more than 20 episodes and a road trip vibe”) and triggers a search. The raw prompt is persisted for history.
Pre-Conditions	User is on the Search page with access to input box.
Post-Conditions	Prompt stored; search request initiated.
Events Flow	1) User types prompt. 2) User presses Enter or clicks “Search.” 3) Prompt saved to history. 4) Request sent to backend.
Alternate Flow and Exceptions	A1) Empty prompt → show validation message. A2) Network loss → cache prompt locally; mark “Pending Sync.”

3.2.2 Use Case: NLP → Structured Query (OpenAI)

Field	Content
NAME OF THE USE CASE	NLP → Structured Query (OpenAI)
Brief Description	System sends prompt to OpenAI and extracts structured fields (genres, years, mood, languages, runtime, cast, rating). Structured query logged for debugging.
Pre-Conditions	Prompt exists and is submitted.
Post-Conditions	Structured query JSON generated and persisted.
Events Flow	1) Backend enqueues NLP call. 2) OpenAI returns structured query. 3) System normalizes fields. 4) Store structured query.
Alternate Flow and Exceptions	A1) OpenAI error → retry/backoff. A2) No structured fields detected → fallback keyword search. A3) Network loss → queue until online.

3.2.3 Use Case: Movie DB Search

Field	Content
NAME OF THE USE CASE	Movie DB Search
Brief Description	System queries local database (CSV/DB) using structured fields. Returns list of candidate movies with metadata.
Pre-Conditions	Structured query JSON exists.
Post-Conditions	Candidate movies returned and cached.
Events Flow	1) Backend composes SQL/DB query. 2) Execute search (genre, year, rating filters). 3) Collect results with title, year, genres, rating, etc. 4) Return to frontend.
Alternate Flow and Exceptions	A1) No results → show “No matches” with fallback suggestions. A2) DB unavailable → error toast; cache structured query for retry.

3.2.4 Use Case: Ranking & Top-N Results

Field	Content
NAME OF THE USE CASE	Ranking & Top-N Results
Brief Description	System sorts candidates (by rating, popularity, recency) and returns top 5 with option to “Show more.”
Pre-Conditions	Candidate list retrieved from DB.
Post-Conditions	Ranked list presented to user.
Events Flow	1) Compute scores. 2) Sort results. 3) Slice top 5. 4) Display results.
Alternate Flow and Exceptions	A1) Ties → break by recency. A2) Less than 5 results → show all with message.

3.2.5 Use Case: Result Cards & Detail View

Field	Content
NAME OF THE USE CASE	Result Cards & Detail View
Brief Description	System displays ranked movie results as clickable cards (poster, title, year, rating, genres, brief overview). Selecting a card opens a detail drawer/page with extended metadata (full overview, runtime, languages, genres, cast snapshot, rating/popularity, optional availability badges) and actions such as **Add to Favorites**.
Pre-Conditions	Ranked candidate list exists from **Movie DB Search/Ranking & Top-N Results**; user is on the Search Results screen.
Post-Conditions	Card grid rendered; if user opens a detail view, detailed metadata is loaded and displayed. If user favorites/unfavorites, Favorites state is updated and persisted.
Events Flow	1) Frontend receives ranked results. 2) Render result cards (poster thumbnail, title/year, rating, genres, truncated overview). 3) User clicks a card → open detail drawer/page. 4) Fetch/expand additional fields if needed (cast snapshot, providers). 5) Display actions: **Add/Remove Favorite**, copy/share link (optional). 6) User clicks **Add to Favorites** → update UI state and persist. 7) User closes detail or navigates back to results.
Alternate Flow and Exceptions	A1) Missing poster → show placeholder image. A2) Long overview → clamp/truncate with “Read more.” A3) Detail fetch error → show non-blocking error message; keep card view usable. A4) Favorite action fails (network/auth) → revert UI state and show retry. A5) Provider data unavailable → hide availability section gracefully. A6) Not logged in (if auth enabled) → store favorite locally and prompt “Sign in to sync” (or proceed in guest mode per product rules). A7) Accessibility: image alt text provided; keyboard focus cycles correctly; Escape closes drawer.

3.2.6 Use Case: Favorites

Field	Content
NAME OF THE USE CASE	Favorites
Brief Description	User adds/removes movies to a personal Favorites list. Favorites are persisted and viewable.
Pre-Conditions	User has search results or detail view open.
Post-Conditions	Favorites updated in profile/storage.
Events Flow	1) User clicks “Add to Favorites.” 2) System updates list. 3) Show confirmation (e.g., heart filled). 4) User can view Favorites page.
Alternate Flow and Exceptions	A1) Duplicate add → ignored. A2) Not logged in (if auth enabled) → store locally until login.

3.2.7 Use Case: Recommendations from Favorites

Field	Content
NAME OF THE USE CASE	Recommendations from Favorites
Brief Description	Based on user’s Favorites, system computes Top 3 similar movies using genre overlap and displays them.
Pre-Conditions	User has ≥1 Favorite saved.
Post-Conditions	Recommendations displayed (excluding already favorited).
Events Flow	1) Collect user Favorites. 2) Compute genre overlap. 3) Rank top matches. 4) Return top 3 recommendations.
Alternate Flow and Exceptions	A1) No Favorites → show empty state message. A2) Similarity returns <3 → show available matches.

3.2.8 Use Case: Availability Badges

Field	Content
NAME OF THE USE CASE	Availability Badges
Brief Description	If availability data exists, system displays streaming/service icons (e.g., Netflix). If not available, section hidden.
Pre-Conditions	Movie metadata includes provider info.
Post-Conditions	Badges shown (if applicable).
Events Flow	1) Check provider data. 2) Render badge(s) on result card.
Alternate Flow and Exceptions	A1) Provider API error → hide section gracefully.

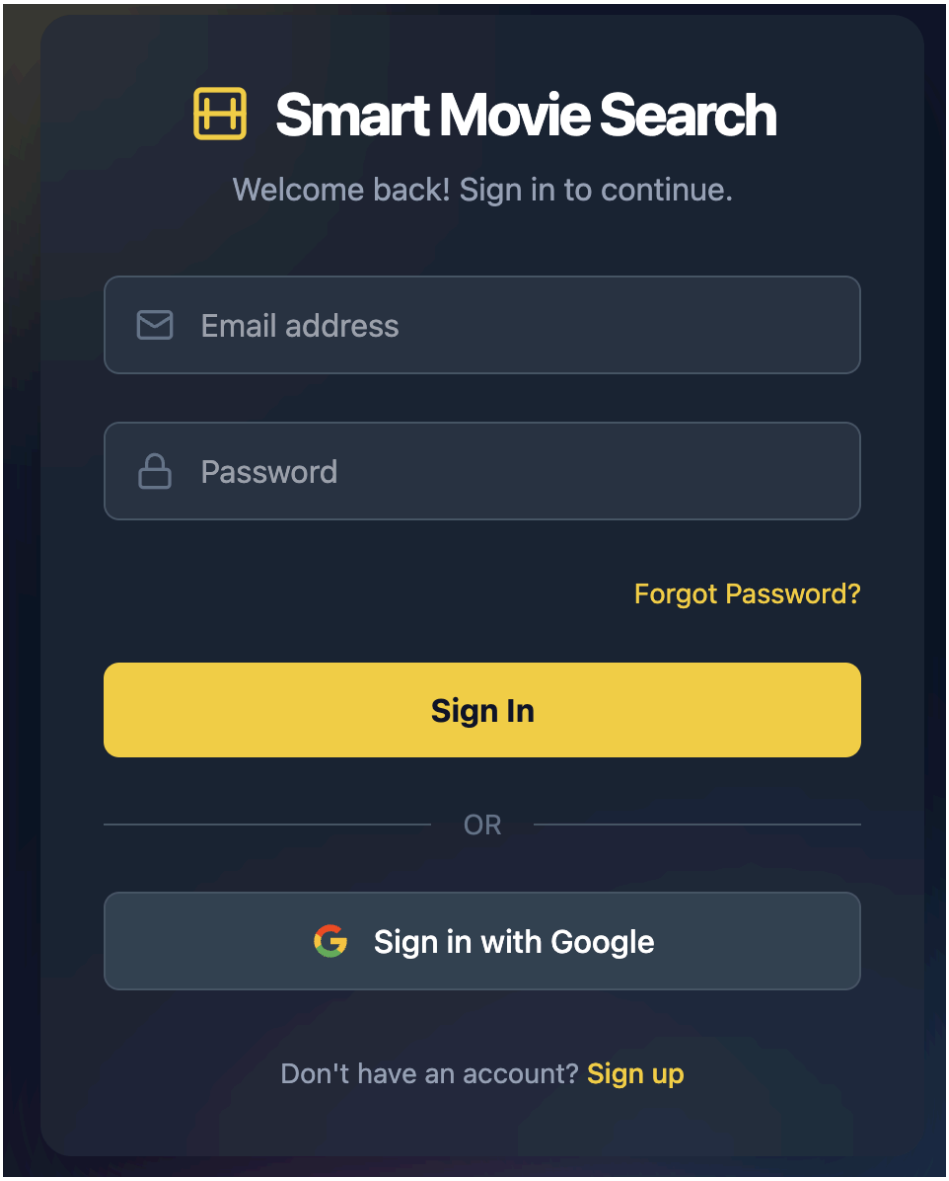
3.2.9 Use Case: Theme Switch

Field	Content
NAME OF THE USE CASE	Theme Switch
Brief Description	User toggles light/dark mode. Choice is persisted across sessions.
Pre-Conditions	User logged in OR local storage available.
Post-Conditions	UI updates immediately; theme persisted.
Events Flow	1) User clicks toggle. 2) System applies CSS theme. 3) Persist choice.
Alternate Flow and Exceptions	A1) Local storage blocked → theme resets on refresh.

3.3 Wireframe

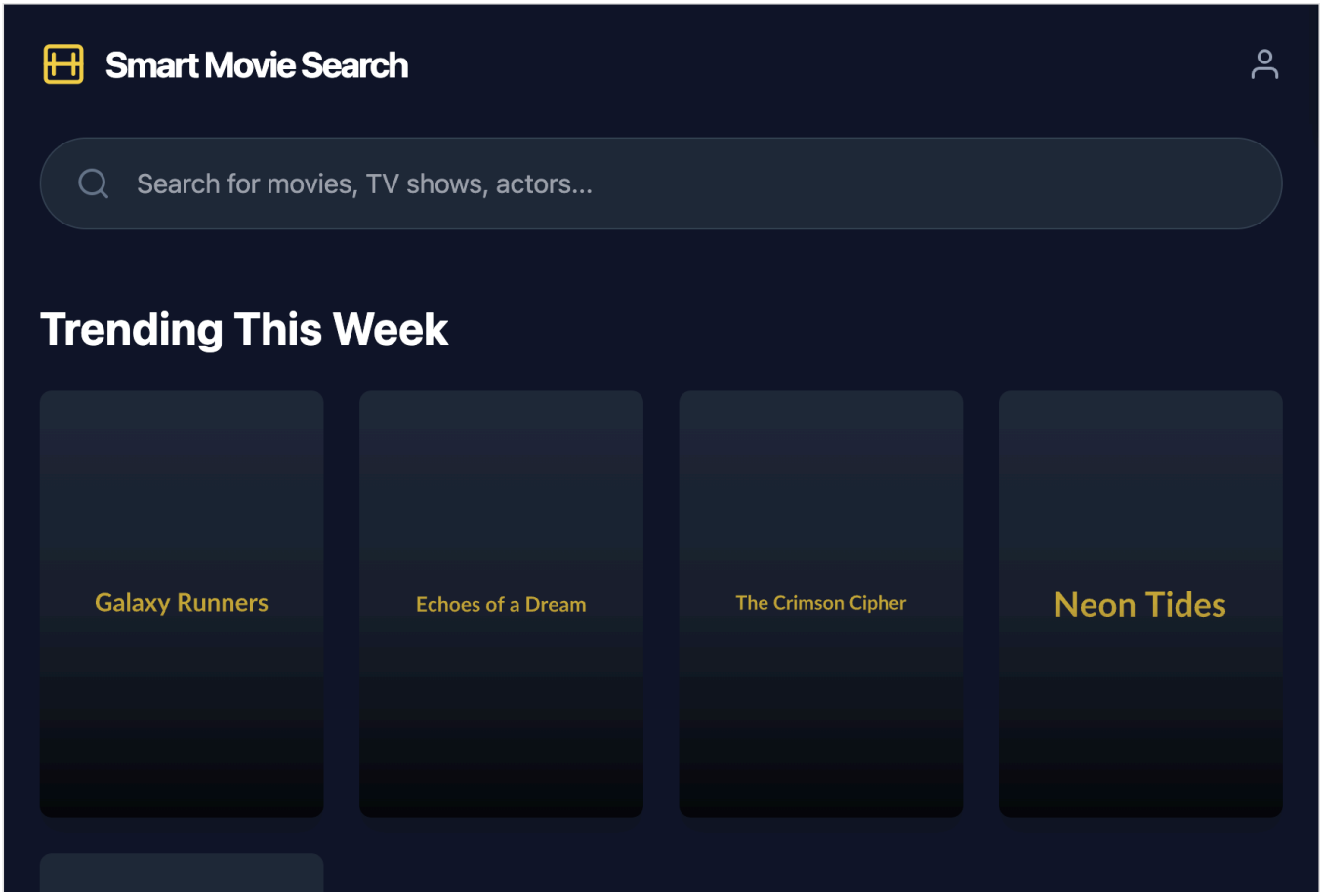
3.3.1 Login Screen

- Basic login fields: Username / Password
- Option for account creation
- Button for Login



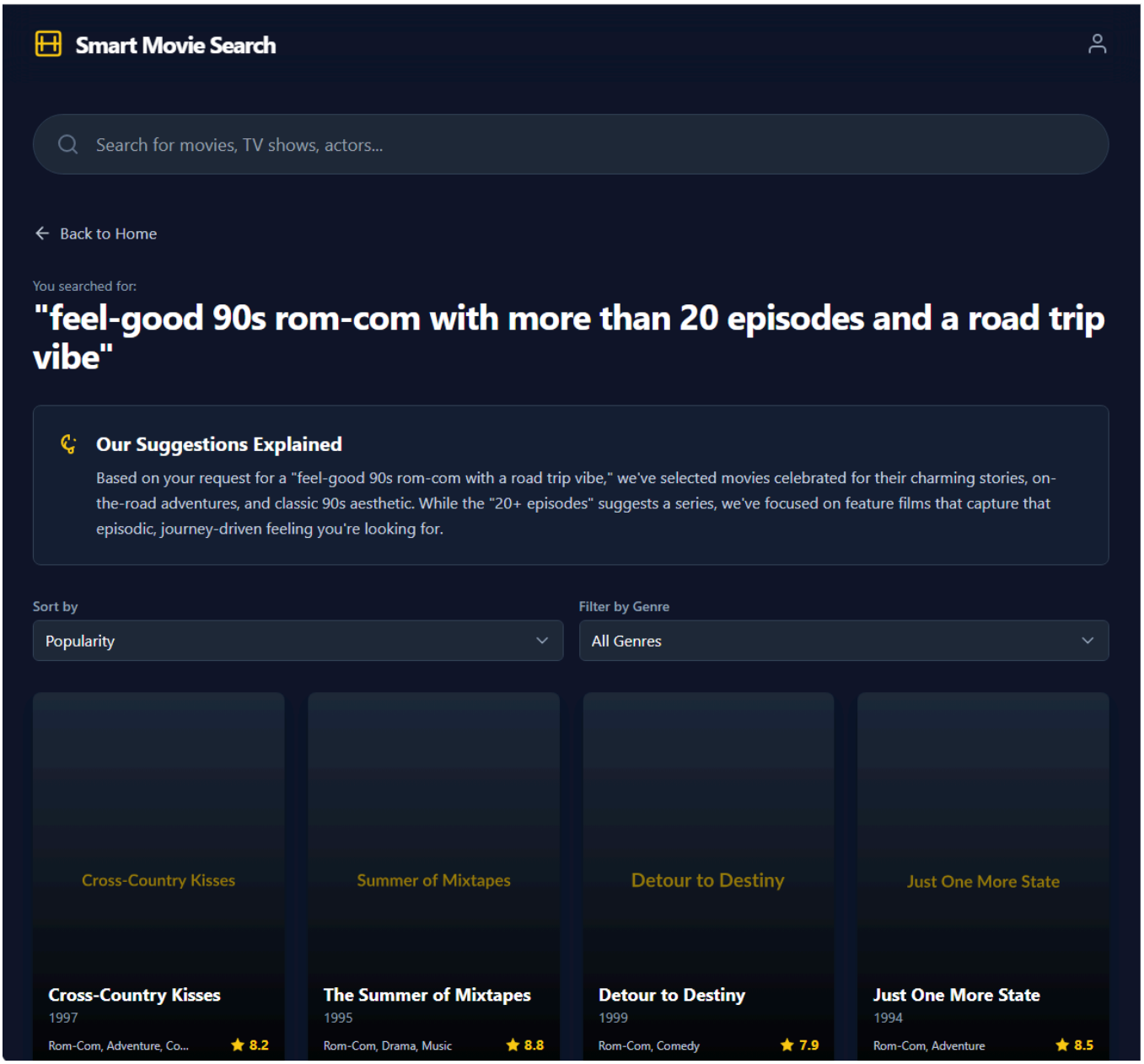
3.3.2 Main Menu

- Search Bar: Central input field for natural-language prompts
- Search Button: Executes query and routes to results page.
- Trending tab: Displays a feed of currently popular or highly-rated movies, refreshed daily/weekly.



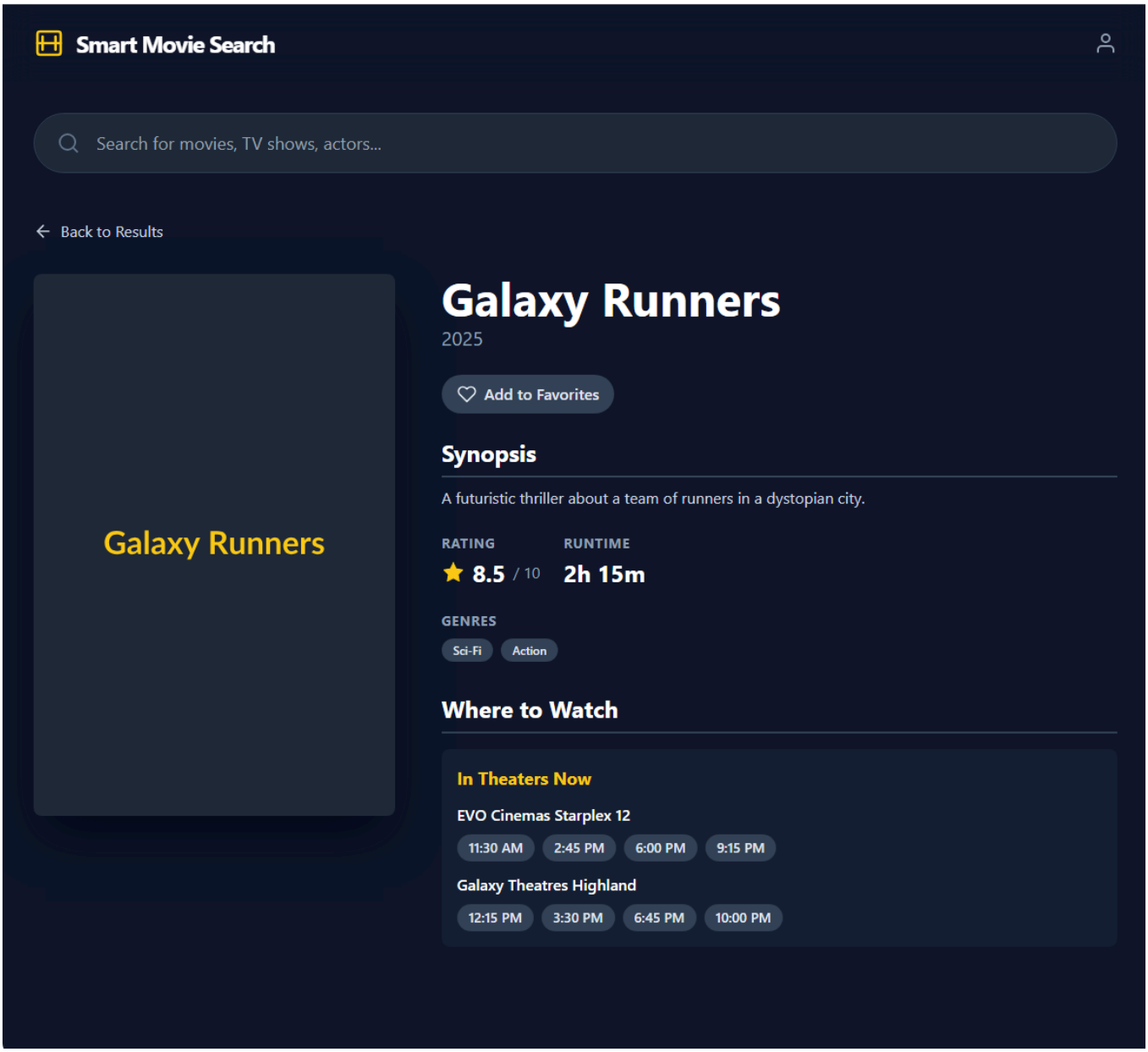
3.3.3 Search Results Page

- Movie Result Cards: Grid of posters with title, year, genres, and rating.
- Show more: Options to show more movies than generated.
- Filters/Sorting: Allow users to filter or sort movies based on attributes such as genre, release year, popularity, or rating.



3.3.4 Movie Detailed View

- Poster and Title: Larger Poster and Basic Movie Info(Title, year)
- Synopsis: Full Description of the movie
- Genres and Rating: Display Categories, Runtime, and Rating
- Add to favorites: An option to add to the Favorites list.



3.4 Hardware/Software Requirements

3.4.1 Hardware Requirements for User

- Device capable of running a modern web browser (desktop, laptop, tablet, or smartphone)
- Minimum **4 GB RAM**
- Minimum **200 MB free disk space** (for cache, local storage, offline features)
- Stable internet connection (required for querying OpenAI API, database, and fetching results)
- Optional: Larger display (desktop/laptop) for richer detail view experience

3.4.2 Software Requirements for User

- **Modern web browser** (latest versions of Chrome, Firefox, Safari, or Edge)
- OR: Mobile platform support via Android/iOS webview or PWA if delivered as a mobile app
- Stable operating system: Windows, macOS, Linux, Android, or iOS

3.4.3 Hardware Requirements for Developer

- Personal computer or laptop with **multi-core processor** (Intel i5/Ryzen 5 or above recommended)
- At least **8 GB RAM** (16 GB preferred if running local DBs or Docker containers)
- Minimum **10 GB free disk space** (for local datasets like MovieLens/IMDb CSVs, logs, and node modules)
- Reliable broadband internet connection (for API access, Git, collaboration tools)

3.4.4 Software Requirements for Developer

- **IDE or code editor**: Visual Studio Code (preferred) or alternatives (WebStorm, IntelliJ, etc.)
- **Version control**: Git + GitHub/GitLab/Bitbucket for repo management
- **UI/UX tools**: Figma, Adobe XD, or similar for wireframes/mockups
- **Frontend stack**: HTML, CSS, JavaScript/TypeScript, React, Tailwind CSS (if chosen)
- **Backend stack**: Node.js (with Express/Nest.js)
- **Database**: PostgreSQL or MongoDB (can start with CSV + DuckDB for MVP)
- **API integrations**: OpenAI API SDK for NLP; optional TMDb/IMDb API keys
- **Testing tools**: Jest, Postman/Insomnia for API testing
- **Browser**: Chrome/Firefox/Safari for testing and debugging
- Optional: **Docker** for containerized development/deployment

4. Non-Functional Requirements

4.1 Reliability

Accurate search results and steady uptime are the main requirements for the system. Essential functions, including database searches, recommendation creation, and natural language search, should all run well. The system should handle failures gracefully (retry/backoff, fallback keyword search, or cached results) if external APIs (such as OpenAI or TMDb) are momentarily unavailable to use.

4.2 Robustness

The application must handle invalid or incomplete input without errors. For example, if a user enters a vague or malformed prompt, the system should still return fallback results instead of crashing. Errors from third-party APIs or missing data (e.g., no poster image, unavailable streaming info) must be handled gracefully with appropriate placeholders.

4.3 Performance

Under typical load conditions, search queries and suggestion results need to be returned in 3–4 seconds. Even when rendering enormous result sets, the frontend should stay responsive (pagination applied if necessary). Particularly during periods of high demand, the system must be scalable to accommodate multiple users without experiencing appreciable performance reduction.

4.4 Maintainability

The system should follow a modular and layered architecture (frontend, backend, database, API integration) so features can be updated independently. Code should be well-documented, follow industry standards (e.g., RESTful API conventions), and support future extensions such as advanced recommendation algorithms, new API integrations, or mobile app versions with minimal rework.

4.5 Security

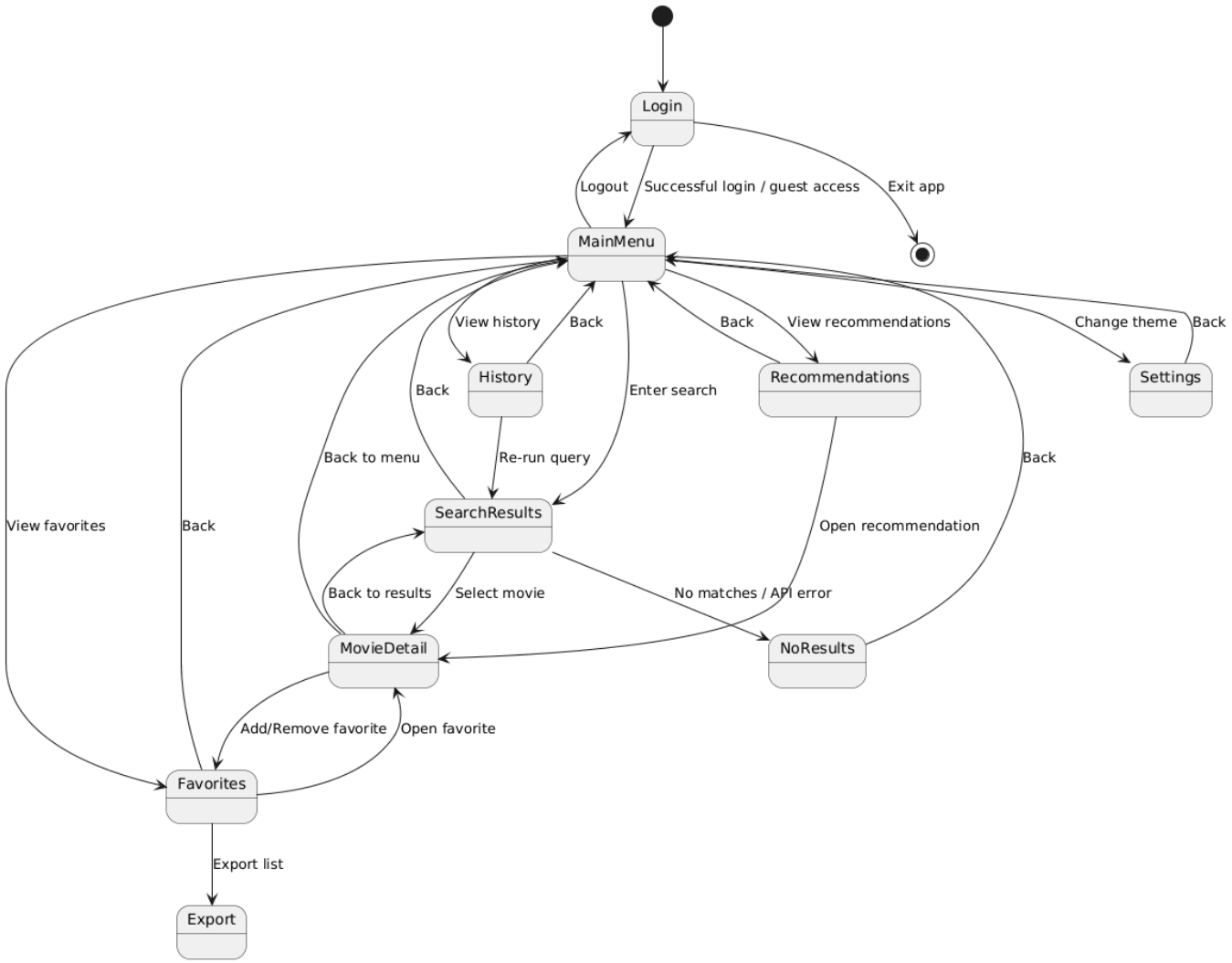
All client-server communications need to be conducted via HTTPS. It is necessary to hash and store user credentials securely if authentication is enabled. Secure tokens must be used in order to access external APIs. The system should guard against typical vulnerabilities like SQL injection, XSS, and CSRF, as well as stop unwanted changes to user data like favorites and history.

4.6 Usability

The user interface needs to be accessible and easy to use. Without training, search, favorites, and recommendations ought to be simple to find and utilize. Screen-reader compatibility, keyboard navigation, and alt text for images are all examples of accessibility guidelines (WCAG 2.1) that should be adhered to. A light/dark theme option and multilingual localization should be supported by the app.

5. Diagrams

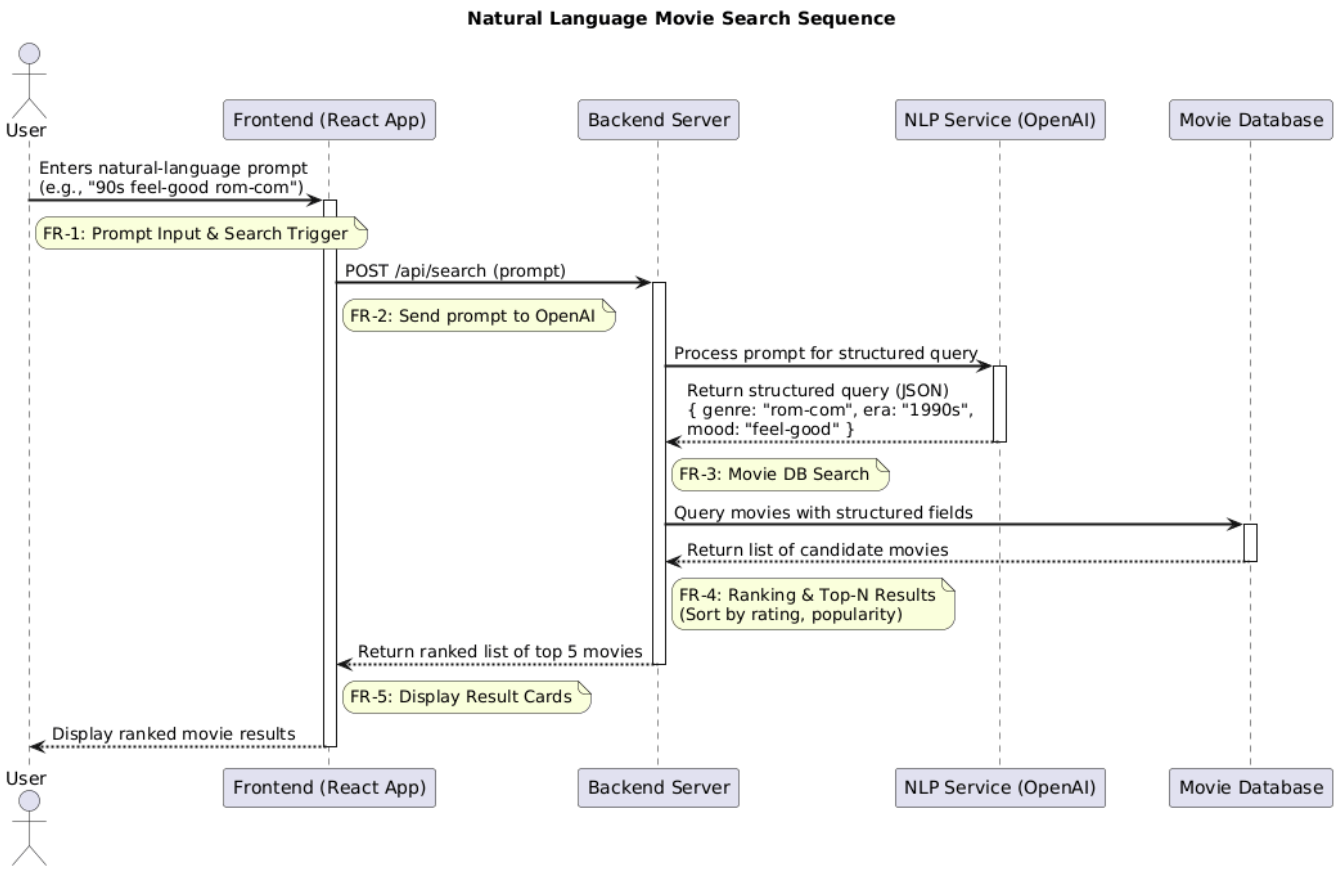
State Diagram



State	Key Events / Actions	Next State(s)	Description
Start	Launch application	Login	Entry point of the system.
Login	Successful login / guest access	Main Menu	Supports optional authentication (FR-9, session handling).
	Exit application	End	Application closed.
Main Menu	Enter natural-language search prompt	Search Results	FR-1 & FR-2: Prompt → Structured Query → DB Search.
	View favorites list	Favorites	FR-6: User's saved movies.

State	Key Events / Actions	Next State(s)	Description
	View recommendations	Recommendations	FR-7: Suggestions based on favorites.
	View history	History	Optional feature: rerun past .prompts
	Change theme	Settings	FR-9: Light/dark mode toggle.
	Logout	Login	Clears session.
Search Results	Select a movie	Movie Detail	FR-5: Opens detailed metadata (overview, cast, rating, availability).
	No matches / API error	No Results	FR-3 fallback: graceful error handling.
Movie Detail	Add / Remove favorite	Favorites (updated)	FR-6: Update Favorites state.
Favorites	Open a favorite	Movie Detail	FR-6: Access full details of saved movies.
	Export favorites (optional)	Export	Optional: share or download favorites list.
Recommendations	Open a recommendation	Movie Detail	FR-7: View detail of a recommended title.
History	Re-run a past query	Search Results	Optional: Executes stored prompt again.
Settings	Toggle light/dark theme	Main Menu	FR-9: Preference persisted locally.
No Results	Acknowledge message	Main Menu	FR-3: User returns to menu after "no matches."

Sequence Diagram



6. References

- PlantUML
- Poore, Mills, and Mutchler , "Planning and Certifying Software System Reliability", IEEE 1993
- Khan, Isfand. "The complete SRS documentation of our developed game." August 24, 2016, https://www.slideshare.net/slideshow/the-complete-srs-documentation-of-our-developed-game/65337752
- GeeksforGeeks, "Non-functional requirements in software engineering," Jan. 3, 2025. https://www.geeksforgeeks.org/non-functional-requirements-in-software-engineering/